

Final Report

Ryan Waterman

Table of Contents

Data Preparation and Feature Engineering3

Modeling and Evaluation.....6

Feature Importance Analysis.....12

Business Recommendations.....15

Appendix.....17

Data Preparation and Feature Engineering

The data preparation and feature engineering process conducted during the final phase of experimentation can be broken into three subsections: data cleaning and preparation, feature engineering, and train-test data split. Each of these subsections are explained, in detail, below.

Data cleaning and preparation was performed experimentally throughout the exploratory model building process. Initially, KNN, QDA, and Logistic Regression were naïvely trained on the entire data set to understand the model performance with the unmodified data set. Stepwise variable selection was utilized to remove non-influential variables from the data set for the KNN, QDA, and Logistic Regression models. The new, advanced models trained during the final phase of experimentation (Decision Tree Classifier and Gradient Boosting Regression) did not utilize stepwise selection. The intent was to determine how well these models “self-select” important variables throughout the training and tuning process, further discussed in the Feature Importance section.

Class imbalance was identified as a key issue for model performance during the exploratory model training process. This was corrected experimentally for each model with a cleaning process that randomly removed (random seed = 42) a given percentage of the observations where Attrition_Flag equaled 0. Each model was trained with configurations of no reduction in Attrition_Flag = 0 observations, 65% reduction of Attrition_Flag = 0 observations, and 75% reduction of Attrition_Flag = 0 observations. See Image 9 in the Appendix for the class imbalance reduction algorithm.

Additionally, outliers made up a significant portion of the data set, described in the bar plot below:

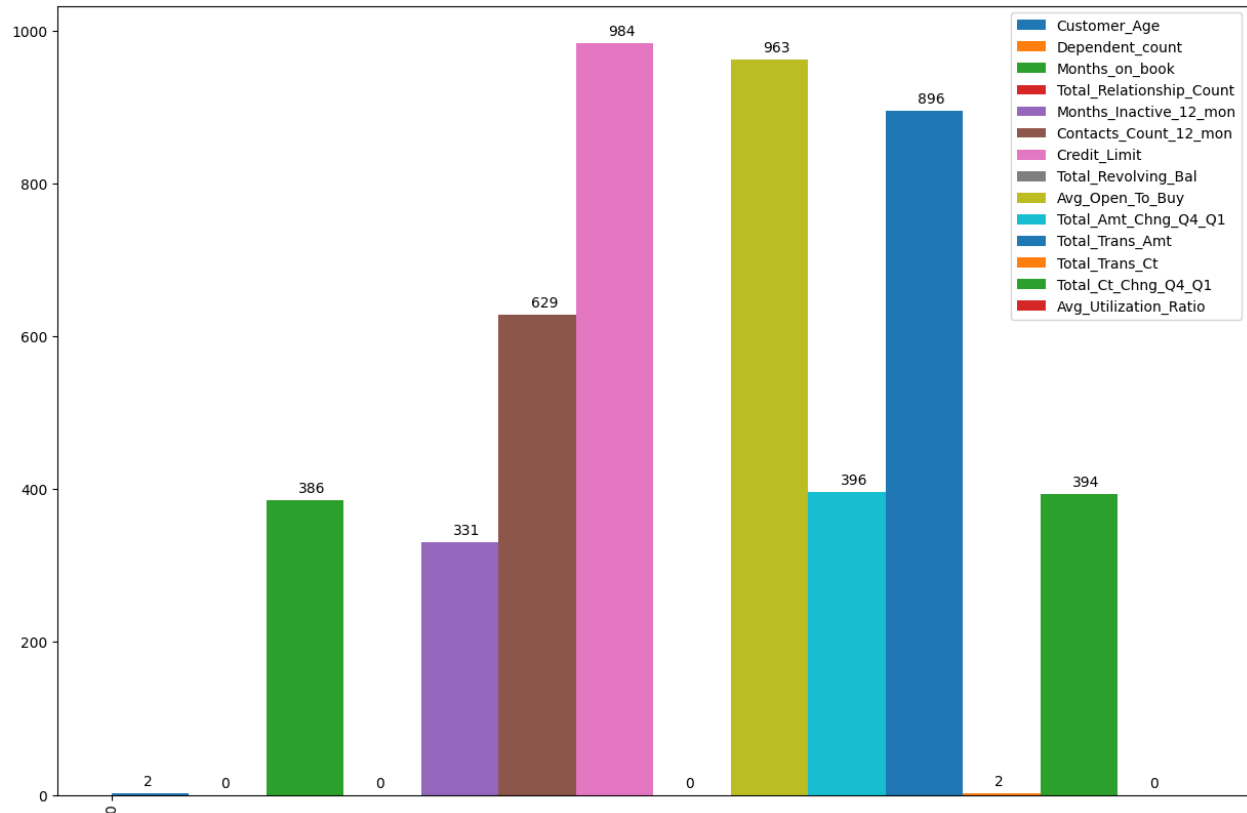


Image 1: Bar Plot of Outlier Count by Quantitative Variable

Outliers were reduced by randomly removing the indices of rows containing outliers across the dataset, invariant of the specific variable. See Images 12 and 13 in the Appendix for the outlier identification and removal algorithms, respectively.

Feature engineering represented a significant effort in this analysis. Harkening back to the Analytic Plan phase of this project, there were four new variables identified for feature engineering: Balance_To_TC, Credit_Limit_per_Income, Leveraged_Balance_Ratio, and Relationship_Strength. A brief description of each variable can be found, below:

1. $\text{Balance_To_TC} = (\text{Total Revolving Balance}) / (\text{Total Transaction Count})$
 - a. A high revolving balance to transaction count ratio would imply the customer is spending large amounts with few transactions, or vice versa, which may have an impact on churn rate.
2. $\text{Credit_Limit_Per_Income} = \text{Credit Limit} / \text{Numerical_Income}$
 - a. The income categories would be reduced to numerical representations. If customers have obtained a high credit limit relative to their income, it can be assumed they have high credit scores, for which the number of open accounts in good standing is a factor.
3. $\text{Leveraged_Balance_Ratio} = (\text{Total Transaction Amount}) / (\text{Average Utilization Ratio} * \text{Credit Limit})$

- a. This feature is an indication of how capable the customer is of paying their balance, normalized to their credit limit.
- 4. Relationship_Strength = Months on Book * Total Relationship Count
 - a. A representation of the strength of the relationship the customer has with the bank. A higher value indicates a stronger relationship, as customers that are heavily invested in the products and services the bank provides may be less likely to stop utilizing the bank.

Lastly, the train-test split selected for the entirety of the analysis was 80% train, 20% test with a random seed of 42. Stratification was done prior to splitting data on the entirety of the data set, as detailed in the class imbalance correction methodology from the data cleaning and preparation section.

The impact of outlier reduction, class imbalance correction, and feature engineering efforts are demonstrated throughout the Modeling and Evaluation section. A test-heavy approach was taken to gain a deeper understanding of how each component of the data cleaning and feature engineering plan impacts model performance.

Modeling and Evaluation

For this phase of analysis, two advanced models were selected for training and testing: Decision Tree Classifier and Gradient Boosting Regression. Both models underwent extensive hyperparameter tuning, in tandem with outlier reduction and class imbalance correction, to identify the optimal training parameters. Upon identifying the optimal training parameters, the model performance was compared, and the feature importance for the respective models were determined.

The Decision Tree Classification (DTC) model was tuned based on the methodology, below:

1. For a given set of input data, find the optimal tree size within the range `depth_min` and `depth_max`.
 - a. The trained model at each depth is tested by predicting on the test data, and the test error is computed.
 - b. The depth where test error is at a minimum would be selected as the optimal depth.
2. Upon identifying the optimal depth, prune the optimized tree using cost complexity pruning to identify an optimal alpha value for a given set of input data.
 - a. Identify a cost complexity pruning path for the optimized tree depth.
 - b. For all alphas in the pruning path, fit a DTC with `max_depth = optimal depth` and `ccp_alpha = alpha`
 - c. Pass test data to pruned DTC with optimal depth and compute test error.
 - d. The alpha value that produced the lowest test error is to be selected as the tuned DTC model.

This framework was used with various combinations of outlier reduction and class imbalance correction. As previously stated, the tuning methodology can be applied to any set of input training and test data. This allows for a further feature tuning step, optimizing the amount of outlier dropout and class imbalance correction to more reliably predict on the test set. With this principle in mind, DTC models were tuned with outlier reductions of 5%, 10%, 15%, 20%, and 25% in combination with class reductions of 50%, 55%, 60%, 65%, 70%, and 75%, resulting in 30 combinations of tuning configurations.

At each step of this approach, metrics such as Accuracy, Recall, Precision, and F1-Score were captured. On average, the DTC tuning method produced the following metrics across all combinations of data cleaning. See Table 10 in the Appendices for the entirety of the tuning results.

Average of Accuracy	Average of Recall	Average of Precision	Average of F1-Score
91.17%	86.78%	86.67%	0.87

Table 1: Average DTC Metrics Across Data Configurations

The best DTC model tuned via this method produced the following metrics at 20% outlier reduction and 50% class imbalance correction:

Best DTC Accuracy	Best DTC Recall	Best DTC Precision	Best DTC F1-Score
92.94%	87.63%	85.00%	0.86

Table 2: Best DTC Metrics Across Data Configurations

The confusion matrix, ROC curve, and AUC for the Best DTC model can be seen in the image below.

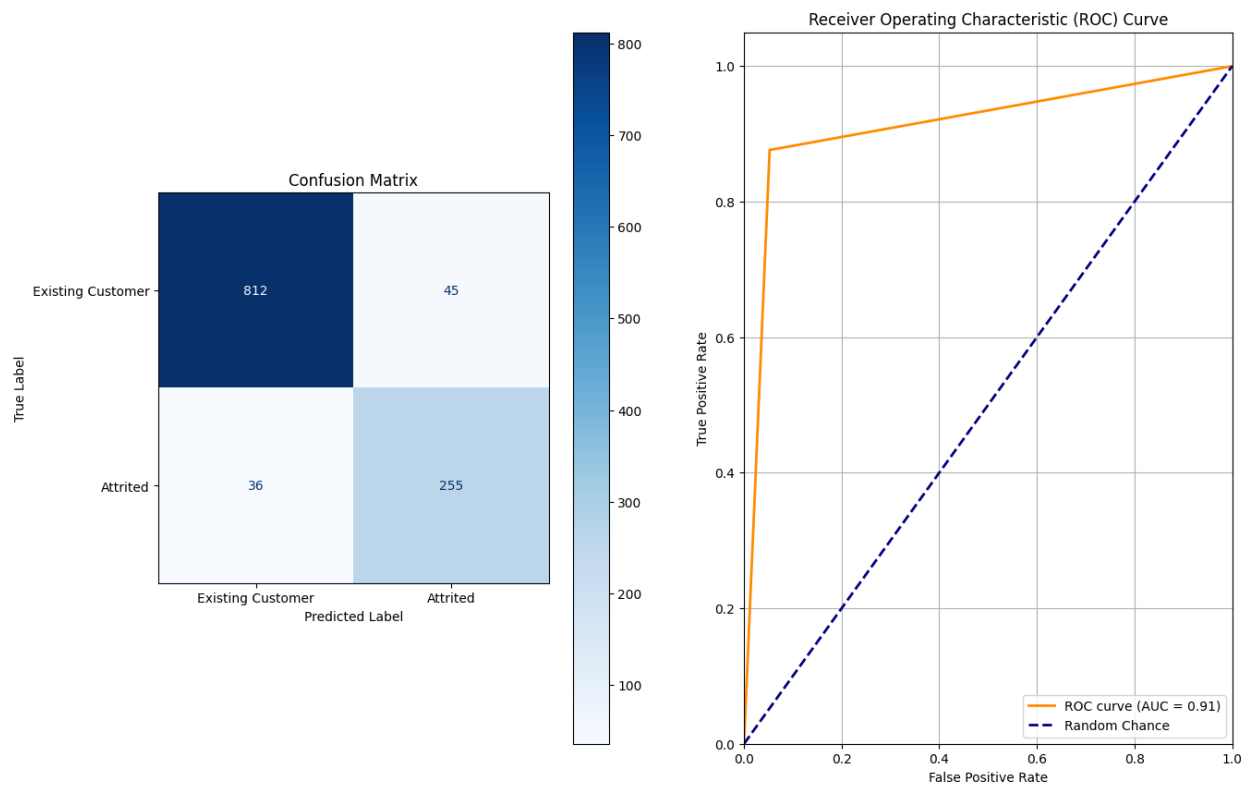


Image 2: Best DTC Confusion Matrix, ROC Curve, and AUC

The Gradient Boosting Regression (GBR) model was tuned using the following methodology:

1. For a given set of input data, iterate through depths in range min_depth to max_depth .
2. For each depth d in range min_depth to max_depth , fit GBR models with learning rate λ in range $\text{min_}\lambda$ to $\text{max_}\lambda$.

- a. The `n_estimators` parameter was fixed at 200.
3. For each combination of depth and learning rate, fit a GBR model to the input training data.
4. Input the test data into the model and compute the test error of the predictions.
5. Identify the combination of depth and learning that produced the lowest test error.

This framework was used with various combinations of outlier reduction and class imbalance correction. As previously stated, the tuning methodology can be applied to any set of input training and test data. This allows for a further feature tuning step, optimizing the amount of outlier dropout and class imbalance correction to more reliably predict on the test set. With this principle in mind, GBR models were tuned with outlier reductions of 5%, 10%, 15%, 20%, and 25% in combination with class reductions of 50%, 55%, 60%, 65%, 70%, and 75%, resulting in 30 combinations of tuning configurations.

At each step of this approach, metrics such as Accuracy, Recall, Precision, and F1-Score were captured. On average, the GBR tuning method produced the following metrics across all combinations of data cleaning. See Table 10 in the Appendices for the entirety of the tuning results.

Average of Accuracy	Average of Recall	Average of Precision	Average of F1-Score
98.04%	96.83%	97.26%	0.97

Table 3: Average of GBR Metrics Across Data Configurations

The best GBR model tuned via this method produced the following metrics at 10% outlier reduction and 70% class imbalance correction:

Best GBR Accuracy	Best GBR Recall	Best GBR Precision	Best GBR F1-Score
99.08%	98.45%	99.07%	0.99

Table 4: Best GBR Metrics Across Data Configurations

The parameters for this model, selected by the tuning process detailed above, were 5 for max depth and 0.06055555555555564 for learning rate. The confusion matrix, ROC curve, and AUC for the Best GBR model can be seen in the image below.

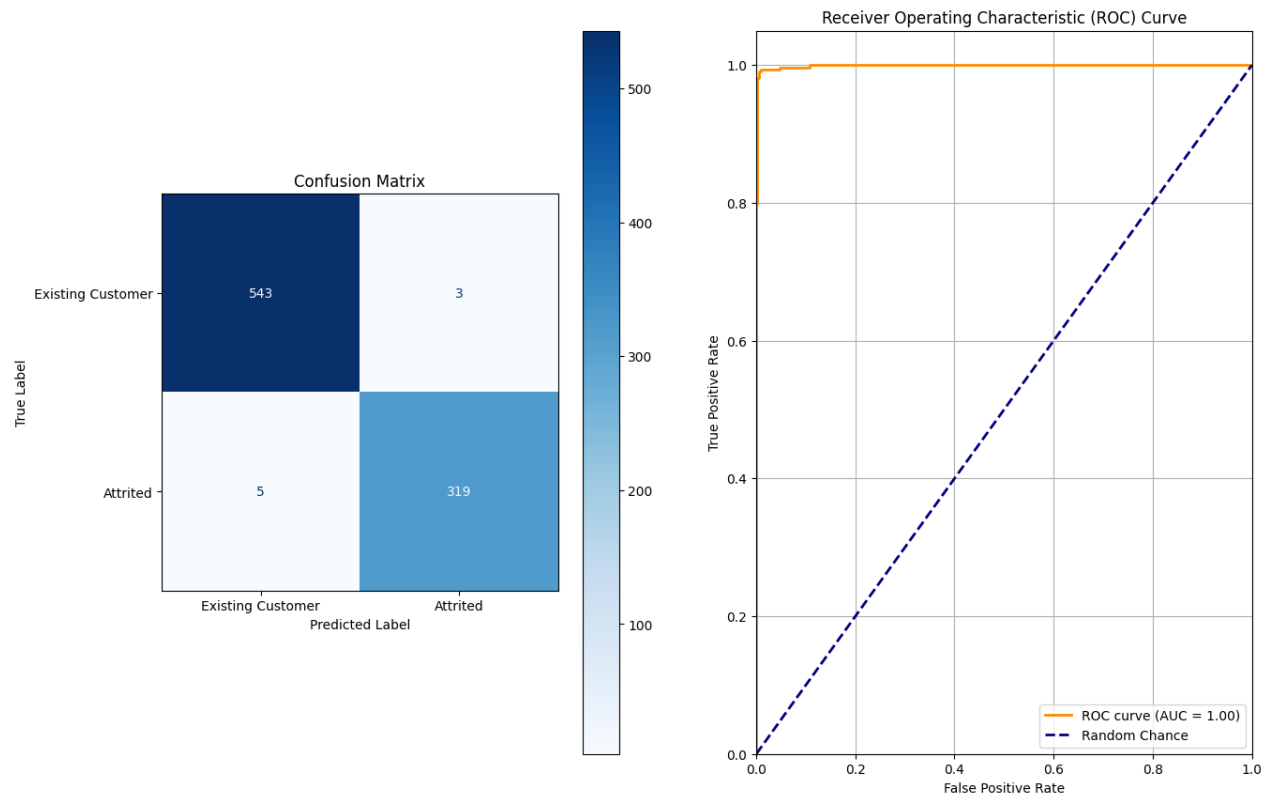


Image 3: Best GBR Confusion Matrix, ROC Curve, and AUC

The results of the GBR model surpassed those of the DTC model by a wide margin, proving the GBR model to be optimal for this application. The differences in average and best model metrics between DTC and GBR can be seen below.

Accuracy	Recall	Precision	F1-Score
6.87%	10.05%	10.59%	0.1

Table 5: Difference in Average Metrics Between GBR and DTC

Accuracy	Recall	Precision	F1-Score
6.14%	10.82%	14.07%	0.13

Table 6: Difference in Best Metrics Between GBR and DTC

Given the large improvement in performance of the GBR model against the DTC, QDA, Logistic Regression, and KNN models, additional testing of the Best GBR model was completed to ensure the model was robust against all configurations of test data.

The Best GBR model was further tested on all 30 configurations of outlier reduction and class reduction. The resulting metrics, averaged across the entirety of data configurations, can be seen below. See Table 11 in the Appendices for the entirety of the test results.

Average of Accuracy	Average of Recall	Average of Precision	Average of F1-Score
97.65%	98.77%	94.31%	0.964333333

Table 7: Average Metrics Across Data Configurations for Best GBR Model

Extrapolated across all test data configurations, the best GBR model continued to outperform even the best DTC, QDA, Logistic Regression, and KNN models. The feature optimized QDA, Logistic Regression, and KNN models, now trained with 25% outlier reduction and 70% class imbalance reduction, produced the following Confusion Matrix and ROC Curve charts:

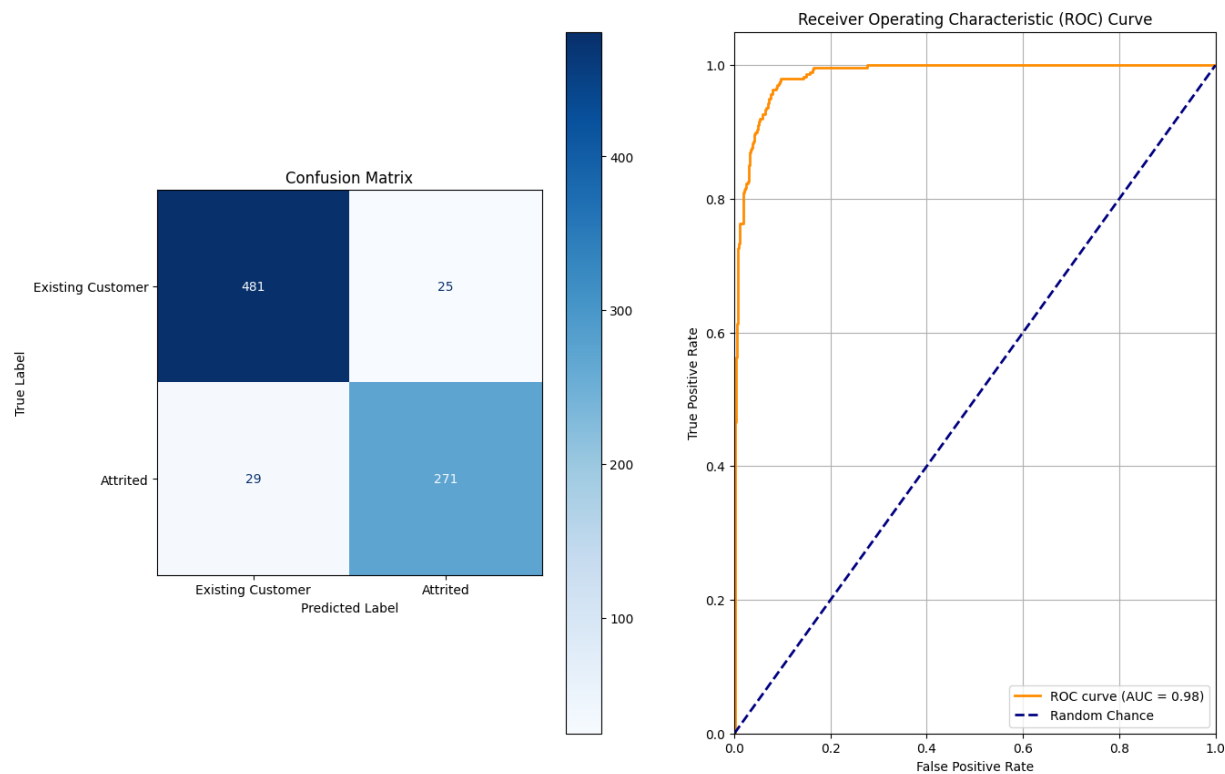


Image 4: Optimized Logistic Regression Model Confusion Matrix and ROC Curve

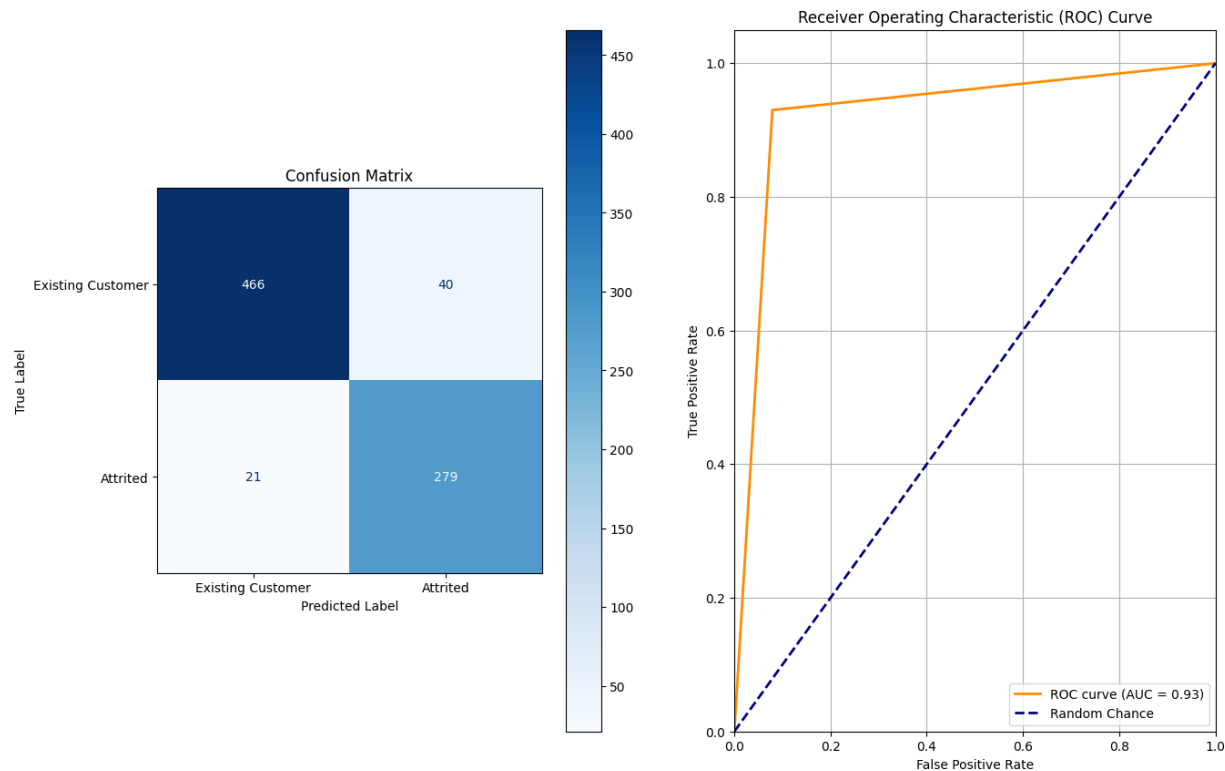


Image 5: Optimized QDA Model Confusion Matrix and ROC Curve

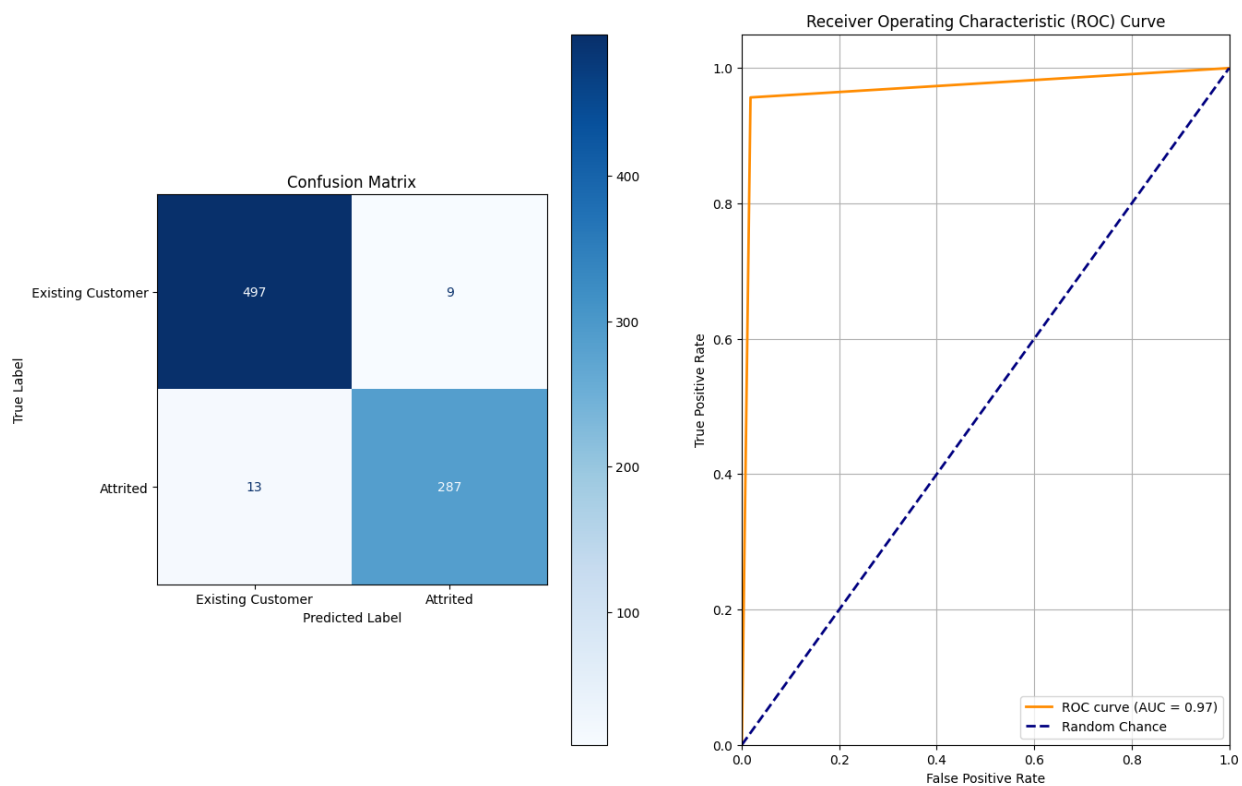


Image 6: Optimized KNN Model Confusion Matrix and ROC Curve

Critical performance metrics for the optimized DTC, GBR, Logistic Regression, QDA, and KNN Models with outlier reduction are compared in Table 8, below:

Model Type	Accuracy	Recall	Precision	F1-Score
Logistic Regression	93.30%	90.33%	91.55%	0.91
QDA	92.43%	93%	87.46%	0.90
KNN	97.27%	95.67%	96.96%	0.96
DTC	92.94%	87.63%	85.00%	0.86
GBR	99.08%	98.45%	99.07%	0.99

Table 8: Optimized Model Performance Metrics

Feature Importance Analysis

Feature selection was not a strategy deployed as part of the DTC and GBR training and tuning process, as a result of the inherent feature selecting structure of tree-based models. The resulting, tuned models were evaluated for feature importance, as seen in Image 7 and Image 8 for GBR and DTC models, respectively.

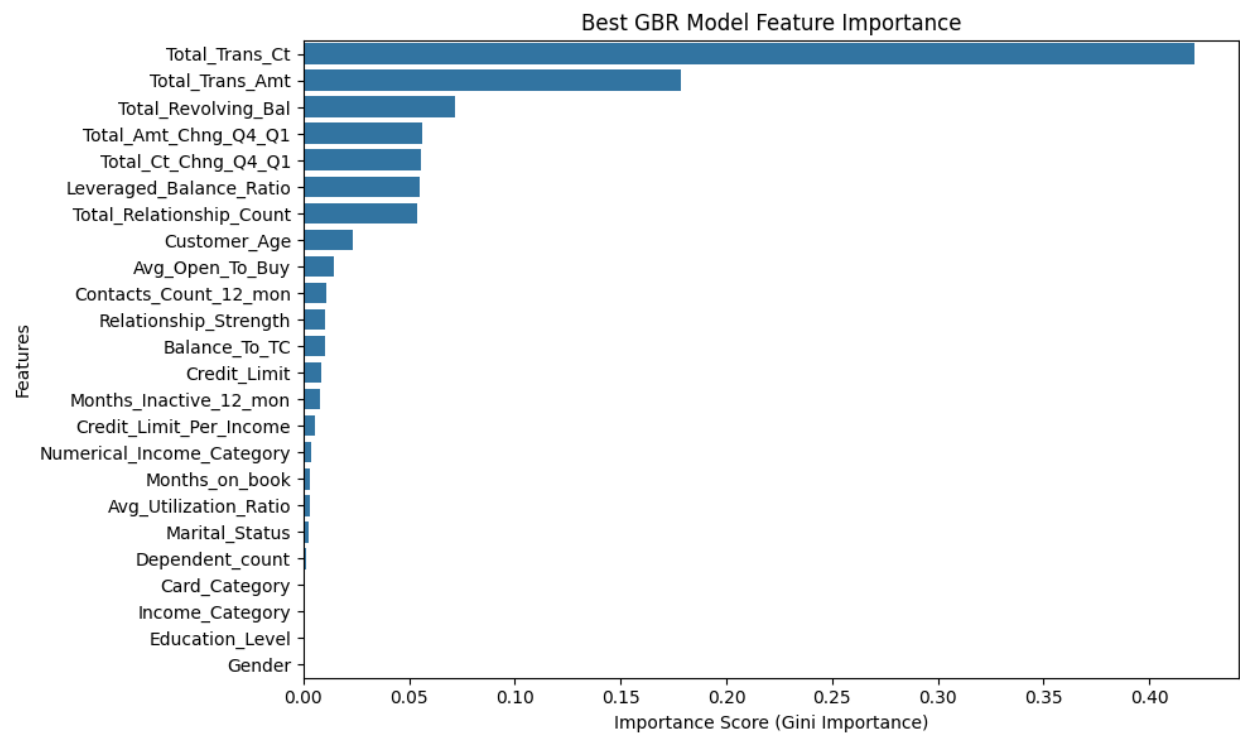


Image 7: Best GBR Model Feature Importance Chart

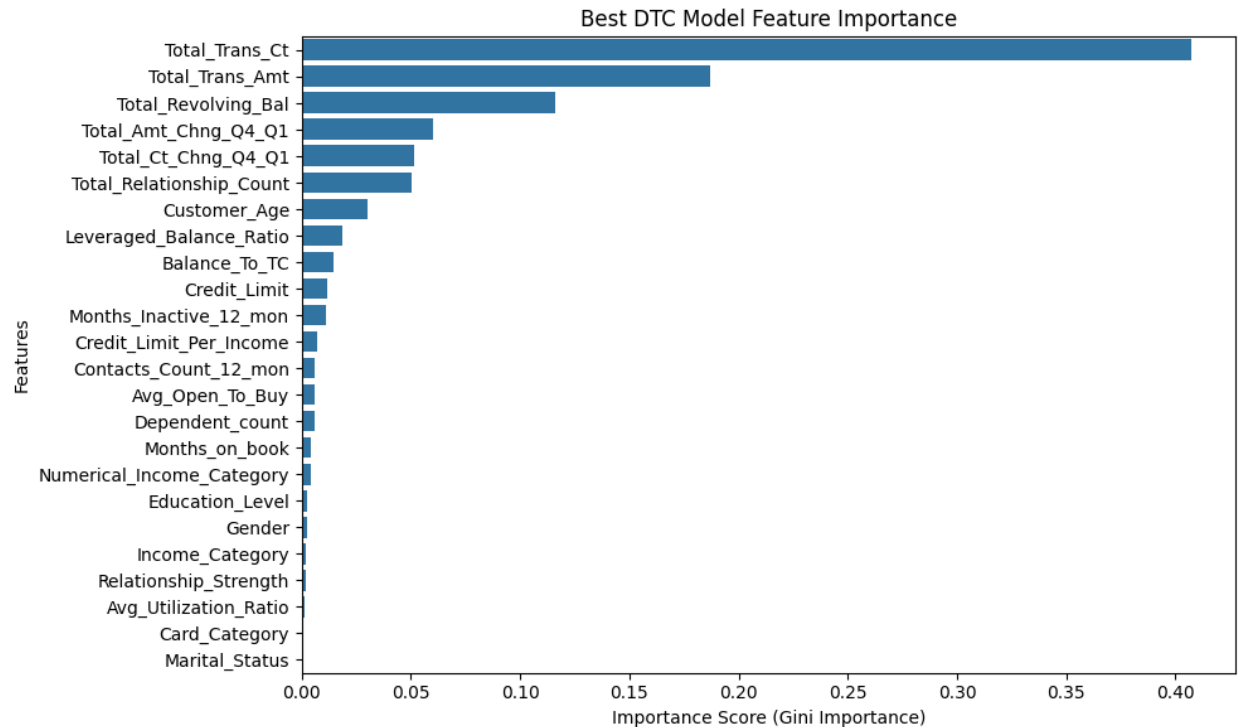


Image 8: Best DTC Model Feature Importance Chart

Both DTC and GBR models identified Total Transaction Count as the most important feature using the Gini Importance metric. Gini Importance evaluates how effectively a given variable can be used to split Attrition_Flag into “Attrited” or “Existing” customers, in this case. Furthermore, the five most important predictors across the best DTC and GBR models are identical, while income category and card category both fell into the bottom 5 predictors. Fundamentally, DTC and GBR models are tree models, although their implementation is different. This similarity in the low-level architecture of both DTC and GBR is likely the reason for such similarity in the selected variables.

Recall that Stepwise Selection was utilized to identify feature importance prior to training the optimized Logistic Regression, QDA, and KNN models. This process selected the features seen in Table 9, below.

Stepwise Selected Variables
Balance To TC
Card Category
Contacts Count 12 mon
Credit Limit Per Income
Dependent count
Education Level
Gender
Leveraged Balance Ratio

Marital Status
Months Inactive 12 mon
Total Amt Chng Q4 Q1
Total Ct Chng Q4 Q1
Total Relationship Count
Total Revolving Bal
Total Trans Amt
Total Trans Ct

Table 9: Stepwise Selected Variables for QDA, KNN, and Logistic Regression Models

Note that `Total_Amt_Chng_Q4_Q1`, `Total_Ct_Chng_Q4_Q1`, `Total_Relationship_Count`, `Total_Revolving_Bal`, `Total_Trans_Amt`, and `Total_Trans_Ct`, the top 5 variables selected by the DTC and GBR models, were also identified as important predictors for the original model architectures. Alternatively, the Stepwise Selection process also selected Marital Status, Gender, Education Level, and Card Category, which the tree-based models unanimously agreed were poor predictors of attrition rate. Despite this difference, the optimized KNN model was able to perform within 3% of the Best GBR model with outlier reduction and class imbalance correction on the dataset.

Considering the nature of the prediction task, there are logical, business-oriented explanations for the feature importance selected by the DTC and GBR models. It is important to remember that a bank is a business like any other, and the best predictor for whether a customer will maintain their relationship is if they use the product. This is the underlying principle of the top 5 predictors, which all directly measure customer engagement, albeit in different capacities.

1. **Total_Trans_Ct** (Total Transaction Count): This is the single most important predictor, reflecting a customer's engagement level across the lifetime of their relationship with the bank. One can infer that a drop in transaction frequency is an indicator of churn.
2. **Total_Trans_Amt** (Total Transaction Amount): Similar to `Total_Trans_Ct`, this feature captures the monetary value of a customer's engagement. Customers willing to transact large quantities of money are clearly invested in the bank.
3. **Total_Revolving_Bal** (Total Revolving Balance): A consistently high balance may indicate a loyal customer, whereas a balance that suddenly drops to zero may signal a balance transfer to a competitor.
4. **Total_Amt_Chng_Q4_Q1** (Change in Transaction Amount): Spending behavior that has changed significantly in the prior quarters may be representative of a strengthening or weakening relationship.
5. **Total_Ct_Chng_Q4_Q1** (Change in Transaction Count): Similar to `Total_Amt_Chng_Q4_Q1`, this feature measures the change in transaction frequency, which is indicative of the likelihood of customer churn.

Business Recommendations

The next stage, upon completion of model training and evaluation, is determining the use case for churn predictions output by the model. There are several attack vectors for bank-customer relations based on churn likelihood. Among the possibilities, targeted retention campaigns for both high and low attrition risk customers, as well as marketing strategies for non-customers, stand out.

The first strategy would utilize the deployed GBR model to predict attrition risk on all existing customers. The benefit of the GBR model is that it can be used to output an attrition probability, which can be used to target the upper and lower percentiles of attrition risk customers. The lowest attrition risk customers could be offered promotions such as bonus points on credit cards, referral bonuses, or access to better products. This would prove to the customer that their decision to trust the bank as a service provider was correct, strengthening the relationship. Similarly, the highest attrition risk customers could be offered lower interest rates or market beating bonus opportunities to incentivize them to continue utilizing the product and eliminate attrition due to competition.

The bank collects several demographic factors for each customer, which is a key insight for new customer outreach. Although most of the demographic factors proved to be inconsequential in the GBR model that strictly predicts churn risk, Age played a role in attrition risk. Age alone is not a strong metric for a marketing campaign, but targeted marketing campaigns can be deployed based on the demographic commonalities of customers that represent the lowest attrition risk. This would require additional analysis, but there is potential for this approach to generate new, low risk customers.

Deployment of the GBR model is the next critical issue. The key considerations in the implementation of this model in production are noted below.

1. **Batch Deployment vs Real-Time Deployment.**
 - a. Real-time deployment is complex. This would require data pipelines to be built in order to feed new observations into the model with the aggregate variables pre-computed. Additionally, identifying observations that may be outliers in a given time frame would be difficult.
 - b. Batch Deployment is more feasible. The model can be run on new observations within a set interval. The output would be a simple list of at-risk customers, which can then be fed to marketing and customer service departments.
2. **Long Term Maintenance:**
 - a. Develop a monitoring dashboard to track the model's performance over time. Tools like Power BI or Tableau can be used to visualize trends in real time.
 - b. Periodically launch programs targeting a small group of at-risk customers to measure the ROI of the retention campaigns.

- c. Identify interval for fine-tuning or retraining. As the bank's customer base grows or customer behavior changes, the model must be re-fit to the new data to continue to be accurate.

The current GBR model is a powerful tool for predicting whether a customer will end their relationship with the bank, however, the reason why must be interpreted by someone that deeply understands the data. Additionally, the data that the model was trained on is static, and future economic factors such as recession risk or competition are not represented.

An opportunity for improvement could be to include more semantic data in the data set, and subsequently the model. Questionnaires providing customer feedback on features such as pricing, customer service, or product specific reviews could provide predictive benefit and empower more pointed campaigns. Additionally, as stated in the long term maintenance suggestions, above, periodic model tuning on new data can help mitigate prediction error due to macroeconomic factors.

Appendices

```
def reduce_data(data, percentage):  
    seed(42)  
    remove_indices = []  
  
    for i in data[data['Attrition_Flag']=='0'].index.values:  
        rand_num = randint(1,100)  
        if rand_num <= percentage:  
            remove_indices.append(i)  
  
    remove = set(remove_indices)  
    all_indices = set(data.index.values)  
  
    reduced_data = data.iloc[list(all_indices - remove)]  
    reduced_data.reset_index(inplace=True)  
    return reduced_data
```

✓ 0.0s Python

Image 9: Class Imbalance Reduction Algorithm

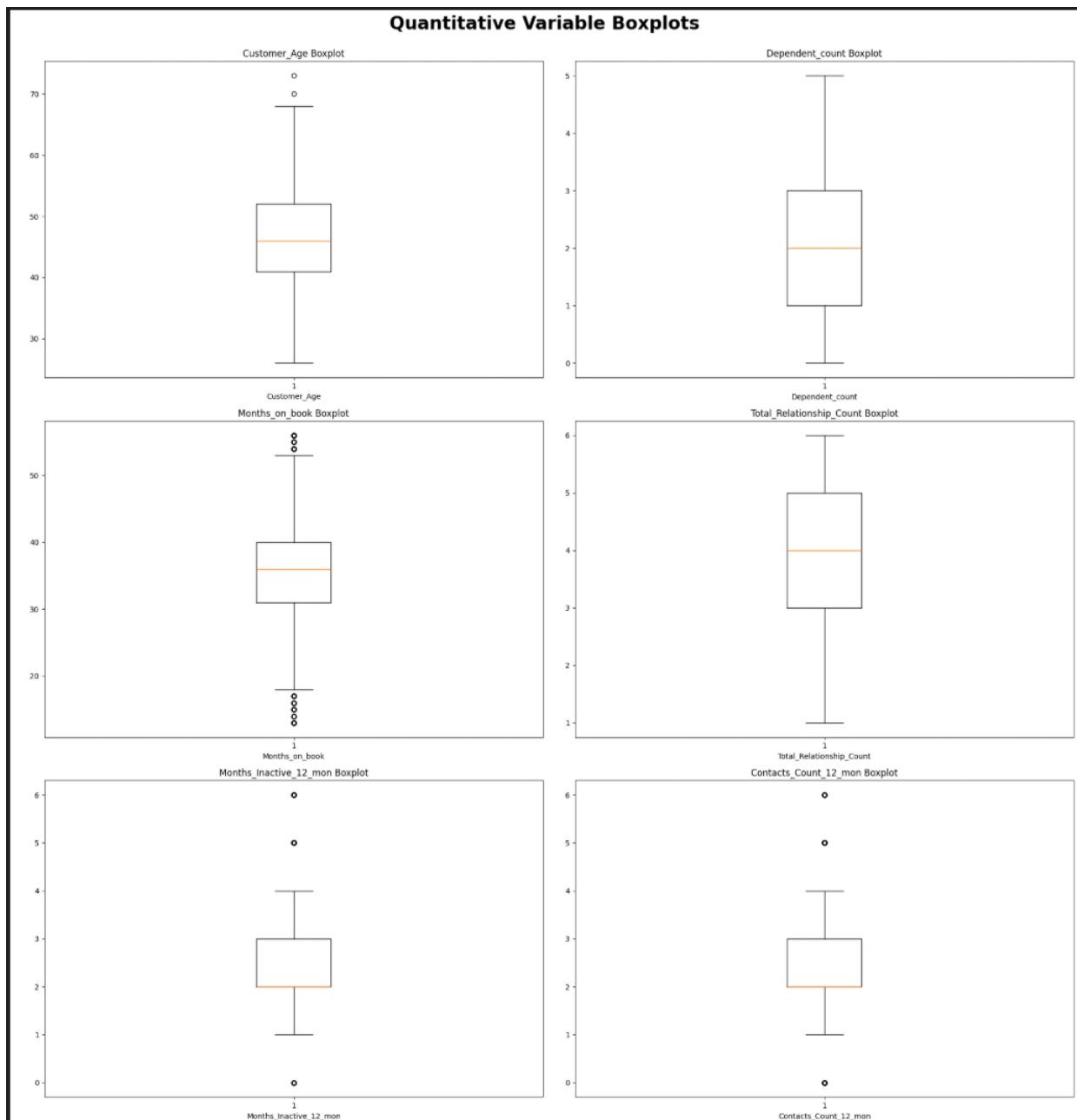


Image 10: Quantitative Variable Box Plots, Part 1

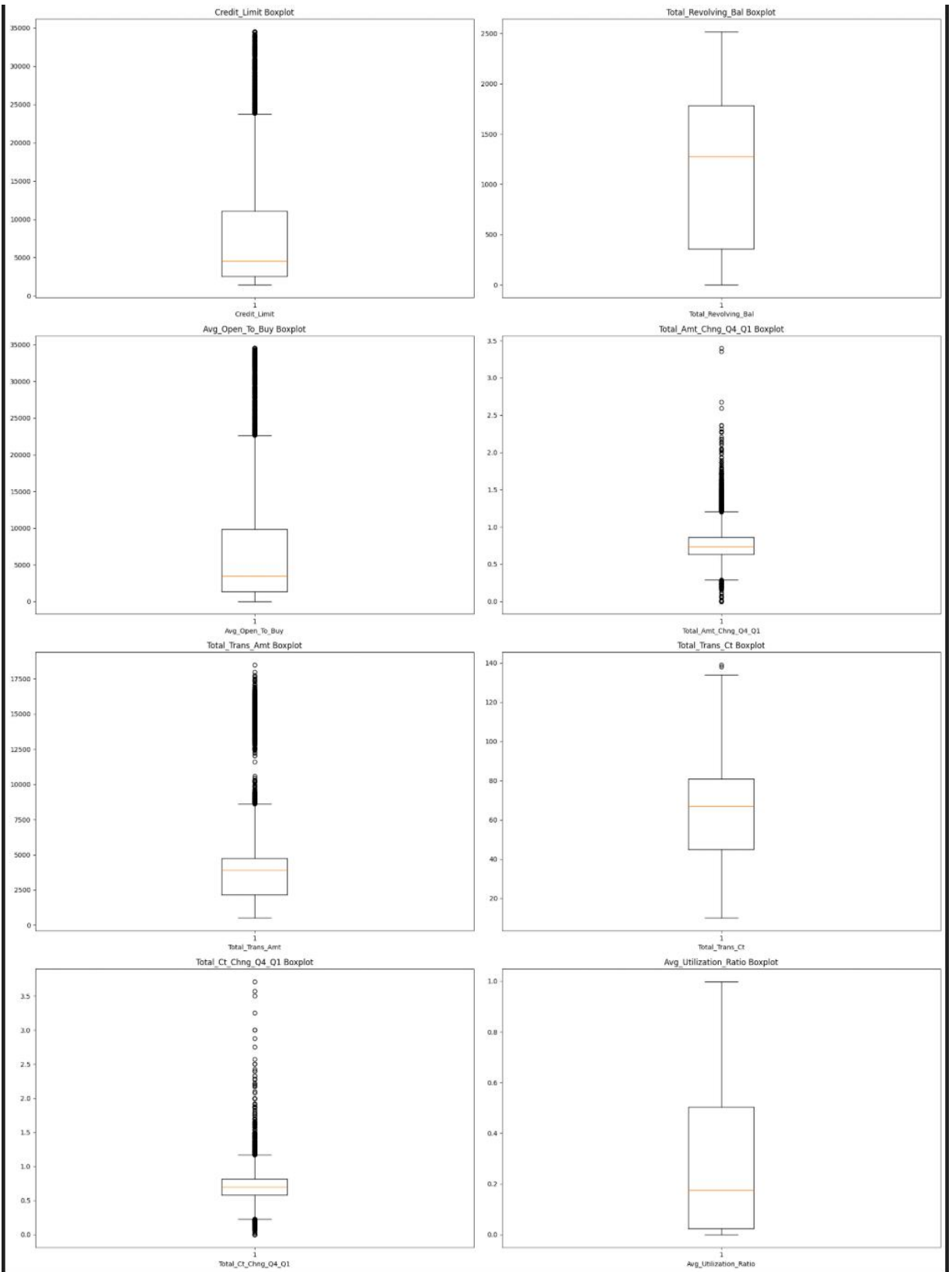


Image 11: Quantitative Variable Box Plots, Part 2

```
outliers = {}

quantitative = list(data.describe().columns.drop('CLIENTNUM'))

for col in quantitative:
    # Calculate Q1, Q3, and IQR
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1

    # Define outlier bounds
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers[col] = list(data[(data[col] < lower_bound) | (data[col] > upper_bound)][col].index)
```

✓ 0.0s Python

Image 12: Outlier Identification Algorithm

```
def remove_outliers(data, total_percentage):
    seed(42)
    reduction_percentage = total_percentage
    remove_indices = []

    outlier_sets = {}
    for key, val in outliers.items():
        outlier_sets[key] = set(val)

    unique_indices = set()

    for index_set in outlier_sets.values():
        unique_indices |= index_set

    unique_indices_list = sorted(list(unique_indices))

    for i in unique_indices_list:
        rand_num = randint(1,100)
        if rand_num <= reduction_percentage:
            remove_indices.append(i)

    remove = set(remove_indices)
    all_indices = set(data.index.values)

    reduced_data = data.iloc[list(all_indices - remove)]
    reduced_data.reset_index(inplace=True)
    return reduced_data
```

✓ 0.0s Python

Image 13: Outlier Reduction Algorithm

Model	Outlier Reduction	Class Reduction	Accuracy	Recall	Precision	F1-Score
Decision Tree Classifier	5	50	92.59%	81.42%	89.15%	0.85
Decision Tree Classifier	10	50	90.64%	84.59%	81.63%	0.83
Decision Tree Classifier	15	50	92.35%	83.44%	87.83%	0.86
Decision Tree Classifier	20	50	92.94%	87.63%	85.00%	0.86
Decision Tree Classifier	25	50	92.07%	85.26%	86.08%	0.86
Decision Tree Classifier	5	55	91.24%	82.44%	88.45%	0.85
Decision Tree Classifier	10	55	92.45%	84.43%	89.81%	0.87
Decision Tree Classifier	15	55	90.44%	83.02%	84.08%	0.84
Decision Tree Classifier	20	55	88.87%	83.70%	80.18%	0.82
Decision Tree Classifier	25	55	91.76%	85.14%	84.23%	0.85
Decision Tree Classifier	5	60	90.99%	85.58%	85.05%	0.85
Decision Tree Classifier	10	60	90.41%	85.15%	82.96%	0.84
Decision Tree Classifier	15	60	92.63%	88.62%	88.62%	0.89
Decision Tree Classifier	20	60	90.66%	84.95%	86.03%	0.85
Decision Tree Classifier	25	60	90.37%	88.56%	82.37%	0.85
Decision Tree Classifier	5	65	91.17%	88.02%	86.73%	0.87
Decision Tree Classifier	10	65	91.02%	88.66%	86.89%	0.88
Decision Tree Classifier	15	65	91.87%	90.15%	86.94%	0.89
Decision Tree Classifier	20	65	90.59%	86.50%	87.04%	0.87
Decision Tree Classifier	25	65	91.66%	86.26%	89.40%	0.88
Decision Tree Classifier	5	70	90.12%	87.80%	86.26%	0.87
Decision Tree Classifier	10	70	92.07%	87.96%	90.48%	0.89
Decision Tree Classifier	15	70	90.82%	88.51%	87.42%	0.88
Decision Tree Classifier	20	70	92.72%	90.70%	89.51%	0.9
Decision Tree Classifier	25	70	90.69%	88.33%	86.89%	0.88
Decision Tree Classifier	5	75	90.45%	88.65%	88.38%	0.89
Decision Tree Classifier	10	75	89.12%	88.14%	85.40%	0.87
Decision Tree Classifier	15	75	90.69%	90.06%	88.99%	0.9
Decision Tree Classifier	20	75	91.33%	89.91%	89.91%	0.9
Decision Tree Classifier	25	75	90.25%	89.75%	88.38%	0.89
Gradient Boosted Regression	5	50	97.83%	95.05%	96.54%	0.96
Gradient Boosted Regression	10	50	98.36%	96.98%	96.98%	0.97
Gradient Boosted Regression	15	50	98.30%	95.94%	97.77%	0.97
Gradient Boosted Regression	20	50	98.78%	97.59%	97.59%	0.98
Gradient Boosted Regression	25	50	97.95%	97.12%	95.58%	0.96

Gradient Boosted Regression	5	55	98.16%	97.45%	96.63%	0.97
Gradient Boosted Regression	10	55	98.20%	96.11%	97.87%	0.97
Gradient Boosted Regression	15	55	96.69%	91.82%	96.69%	0.94
Gradient Boosted Regression	20	55	98.21%	96.87%	97.17%	0.97
Gradient Boosted Regression	25	55	98.35%	97.10%	96.75%	0.97
Gradient Boosted Regression	5	60	98.18%	97.18%	96.88%	0.97
Gradient Boosted Regression	10	60	97.75%	96.70%	95.75%	0.96
Gradient Boosted Regression	15	60	97.91%	96.31%	97.20%	0.97
Gradient Boosted Regression	20	60	98.88%	98.12%	98.43%	0.98
Gradient Boosted Regression	25	60	97.93%	97.06%	96.43%	0.97
Gradient Boosted Regression	5	65	98.03%	97.01%	97.30%	0.97
Gradient Boosted Regression	10	65	97.47%	96.22%	96.78%	0.97
Gradient Boosted Regression	15	65	97.65%	95.69%	97.49%	0.97
Gradient Boosted Regression	20	65	97.37%	94.79%	97.78%	0.96
Gradient Boosted Regression	25	65	98.55%	98.40%	97.47%	0.98
Gradient Boosted Regression	5	70	98.09%	97.62%	97.33%	0.97
Gradient Boosted Regression	10	70	99.08%	97.62%	99.07%	0.99
Gradient Boosted Regression	15	70	98.24%	96.27%	99.04%	0.98
Gradient Boosted Regression	20	70	98.18%	97.67%	97.35%	0.98
Gradient Boosted Regression	25	70	98.01%	97.33%	97.33%	0.97
Gradient Boosted Regression	5	75	97.45%	97.24%	96.65%	0.97
Gradient Boosted Regression	10	75	98.45%	98.72%	97.47%	0.98
Gradient Boosted Regression	15	75	98.01%	97.59%	97.89%	0.98

Gradient Boosted Regression	20	75	97.02%	96.53%	96.53%	0.97
Gradient Boosted Regression	25	75	98.21%	97.83%	98.13%	0.98

Table 10: Results of All DTC and GBR Tuning Iterations

Outlier Reduction	Class Reduction	Accuracy	Recall	Precision	F1-Score
5	50	96.22%	99.07%	87.91%	0.93
10	50	97.87%	98.79%	93.70%	0.96
15	50	97.45%	99.06%	92.15%	0.95
20	50	97.04%	97.94%	91.05%	0.94
25	50	97.06%	98.72%	91.39%	0.95
5	55	96.85%	98.87%	91.60%	0.95
10	55	98.74%	99.40%	96.51%	0.98
15	55	97.70%	98.74%	93.73%	0.96
20	55	96.89%	99.69%	90.86%	0.95
25	55	97.58%	98.91%	92.54%	0.96
5	60	97.03%	99.06%	91.86%	0.95
10	60	98.24%	98.68%	95.53%	0.97
15	60	98.51%	99.38%	96.13%	0.98
20	60	97.77%	98.75%	94.59%	0.97
25	60	97.72%	99.02%	94.10%	0.96
5	65	96.88%	98.80%	92.70%	0.96
10	65	98.73%	98.84%	97.70%	0.98
15	65	97.11%	98.46%	93.57%	0.96
20	65	98.25%	99.08%	96.13%	0.98
25	65	98.22%	98.72%	96.26%	0.97
5	70	97.87%	99.70%	94.90%	0.97
10	70	95.63%	94.75%	93.60%	0.94
15	70	98.24%	98.45%	96.94%	0.98
20	70	96.84%	98.34%	93.38%	0.96
25	70	98.39%	99.00%	96.74%	0.98
5	75	97.58%	98.77%	95.55%	0.97
10	75	99.22%	98.40%	99.68%	0.99
15	75	98.54%	99.40%	97.35%	0.98
20	75	97.56%	99.05%	95.44%	0.97
25	75	97.80%	99.38%	95.81%	0.98

Table 11: Best GBR Performance Across All Test Data Configurations